

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration : 1 Hour

Marks:50

Total

Annexure A

Analytical Problem Solving

Code of Conduct:

I **T. Niranjana Kumar (192473007)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: T. Niranjana Kumar

1. Water Jug Problem (4-Gallon and 3-Gallon Jugs)

Initial State

- 4-gallon jug = 0
- 3-gallon jug = 0

Steps

Step	4-Gallon Jug	3-Gallon Jug	Action
1	0	3	Fill the 3-gallon jug.
2	3	0	Pour all water into the 4-gallon jug.
3	3	3	Fill the 3-gallon jug again.
4	4	2	Pour water into the 4-gallon jug until it is full. (Only 1 gallon is transferred, leaving 2 gallons in the 3-gallon jug.)
5	0	2	Empty the 4-gallon jug.
6	2	0	Pour the remaining 2 gallons from the 3-gallon jug into the 4-gallon jug.

Final Answer

- 4-gallon jug = 2 gallons
- 3-gallon jug = 0 gallons

2. Mars Rover Agent

i. Percepts

The Mars Rover receives:

- Camera images
- Rock and soil data
- Temperature
- Weather conditions
- Obstacle information
- Battery level
- Position (GPS-like navigation)

ii. Operating Environment

- **Partially Observable**
- **Dynamic**
- **Sequential**
- **Unknown**
- **Continuous**

iii. Actions

- Move forward
- Turn left/right
- Stop
- Capture images
- Collect rock/soil samples
- Analyze samples
- Send data to Earth
- Avoid obstacles
- Recharge using solar panels

iv. Performance Evaluation

- Number of samples collected
- Accuracy of analysis
- Distance covered
- Energy efficiency
- Successful obstacle avoidance
- Data transmitted successfully

v. Suitable Agent Architecture

Learning Agent

Reason:

A Learning Agent improves its performance by learning from previous experiences, adapting to the unknown Martian environment, avoiding obstacles, and selecting better routes.

3. 8-Queens Problem

Problem Formulation

Initial State

- Empty chessboard.

State

- Any arrangement of queens on the board.

Actions

- Place one queen in the next column.
- Ensure no two queens attack each other.

Goal State

- Place all **8 queens** so that:
 - No two queens are in the same row.
 - No two queens are in the same column.
 - No two queens are on the same diagonal.

Path Cost

- Each queen placement has a cost of 1.
- Total cost = 8 placements.

Search Strategy

- **Backtracking Search**
- **Depth-First Search (DFS)**

Example Solution

Q.....

....Q...

.....Q

.....Q..

..Q.....

.....Q.

.Q.....

...Q.....

4. OLA Cab Booking Problem

Type of Problem-Solving Agent

Goal-Based Agent

Reason:

The goal of the customer is to reach the destination. The agent compares different cab options (Mini, Micro, Sedan, Prime, Shared, etc.) and selects the best one based on the user's comfort, availability, and cost.

Pseudocode

Algorithm OLA_Cab_Booking

Start

Input Pickup Location

Input Destination

Display available cab types

(Mini, Micro, Sedan, Prime, Shared)

User selects cab type

Check cab availability

If cab is available

 Calculate fare

 Confirm booking

 Assign nearest driver

 Display driver details

 Start trip

Else

 Display "Cab Not Available"

End If

End

Flow

Start



Enter Pickup & Destination



Show Available Cabs



Select Cab Type



Cab Available?



Yes → Calculate Fare → Confirm Booking → Assign Driver → Start Trip



No → Display "Cab Not Available"



End

5. Uniform Cost Search (UCS) Solution

From the given graph:

- $S \rightarrow A = 1$
- $S \rightarrow G = 12$
- $A \rightarrow B = 3$
- $A \rightarrow C = 1$
- $B \rightarrow D = 3$
- $C \rightarrow D = 1$
- $C \rightarrow G = 2$
- $D \rightarrow G = 3$

Step 1: Initial State

- Start Node = **S**
- Goal Node = **G**

Priority Queue:

Queue Cost

S 0

Step 2: Expand S

Possible paths:

- $S \rightarrow A = 1$
- $S \rightarrow G = 12$

Queue:

Queue Cost

A 1

G 12

Step 3: Expand A (Lowest Cost)

Possible paths:

- $S \rightarrow A \rightarrow B = 1 + 3 = 4$
- $S \rightarrow A \rightarrow C = 1 + 1 = 2$

Queue:

Queue Cost

C 2

B 4

G 12

Step 4: Expand C

Possible paths:

- $S \rightarrow A \rightarrow C \rightarrow D = 2 + 1 = 3$
- $S \rightarrow A \rightarrow C \rightarrow G = 2 + 2 = 4$

Queue:

Queue Cost

D	3
B	4
G	4
G	12

Step 5: Expand D

Possible path:

- $S \rightarrow A \rightarrow C \rightarrow D \rightarrow G = 3 + 3 = 6$

Queue:

Queue Cost

B	4
G	4
G	6
G	12

Step 6: Expand B

Possible path:

- $S \rightarrow A \rightarrow B \rightarrow D = 4 + 3 = 7$

Already have **D** = 3, so ignore.

Queue:

Queue Cost

G	4
G	6

Queue Cost

G 12

Step 7: Expand Goal

The first goal node removed from the queue has cost **4**.

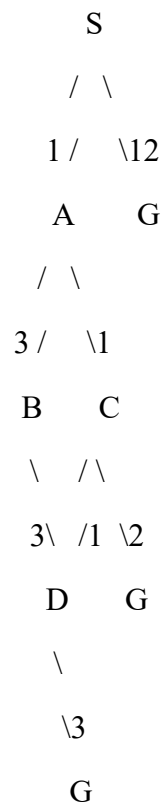
Least Cost Path

S → A → C → G

Total Cost

1 + 1 + 2 = 4

Search Tree



Pseudocode of Uniform Cost Search (UCS)

UniformCostSearch(Start, Goal)

1. Create a Priority Queue PQ
2. Insert (Start, Cost = 0)

3. Mark Start as visited

4. While PQ is not empty

 Remove node with minimum cost

 If node == Goal

 Return path and cost

 For each neighboring node

$\text{NewCost} = \text{CurrentCost} + \text{EdgeCost}$

 If neighbor is not visited

 Insert into PQ with NewCost

5. Return Failure